

CSE 220: Signals and Linear Systems

Predicted Lab Exam Questions

Practice Set on Signals and Their Properties

Topics: Time Reversal • Time Shift • Time Scaling • Phase Shift • Even–Odd Decomposition • Energy/Power • Linearity • Time Invariance • Causality

This document is a set of **predicted practice questions** modeled on the format, difficulty, and structure of the CSE 220 onlines. It is generated for revision purposes and is **not** an official exam paper. Each question mirrors the style of the real onlines: you implement functions on discrete signals (or sampled continuous signals) using NumPy, then verify and explain your results. Six independent question sets are provided, each sized for a 30–35 minute online worth 10 marks.

Discrete-signal convention (same as all CSE 220 onlines): A discrete signal is a NumPy array. Signals extend from $-\infty$ to ∞ , but the array stores only the values for $n = -8 \dots 8$ ($INF = 8$, length 17). Any index outside $-8 \dots 8$ is treated as 0. The example array $x = [0,0,0,0,0,0,0.5,2,1,0.5,1,0,0,0,0,0]$ represents a signal with $x[-1]=2$, $x[0]=1$, $x[1]=0.5$, $x[-2]=0.5$, $x[2]=1$.

General reminders (hard-won from the onlines): (1) A plain reversal $x[::-1]$ is only correct when the index axis is symmetric about 0 — which it is here, since n runs $-8 \dots 8$. (2) Use zero-padded shifts, not `np.roll`, for a true (non-circular) delay. (3) For discrete scaling, never interpolate onto non-integer indices — those samples are 0. (4) Bonus marks are given for fully vectorized NumPy solutions with no explicit Python loops.

Question Set 1 — Time Shift and Time Scaling

Total: 10 Marks | Suggested Time: 30 min

You are provided with a template that stores a discrete signal as a length-17 NumPy array ($n = -8 \dots 8$). Implement the two functions below.

Task 1 — `time_shift_signal(x, k)`

Return the shifted signal $x[n - k]$ as a NumPy array of the same length. k is an integer with $-8 \leq k \leq 8$. Positive k delays (shifts right); negative k advances (shifts left). Values shifted out of the $-8 \dots 8$ window are dropped; vacated positions are filled with 0. *Do not use `np.roll`* (it wraps values around, which is wrong).

Task 2 — `time_scale_signal(x, k)`

Return the compressed signal $x[kn]$ for a positive integer k . Output position n takes the value of input index kn ; if kn falls outside $-8 \dots 8$, the output is 0. Example: for $k = 2$, output $n = -1, 0, 1$ draw from input indices $-2, 0, 2$.

Bonus: Solve both without explicit Python loops.

Task 1 (time shift)	4
Task 2 (time scaling $x[kn]$)	4
Bonus (vectorized, no loops)	2

Verification you should run: confirm $x[n-0]$ returns the original; confirm shifting right by k then left by k restores the original (up to samples pushed off the edge); for $x[2n]$, confirm only samples at even input indices survive.

Question Set 2 — Time Reversal and Even–Odd Decomposition

Total: 10 Marks | Suggested Time: 30 min

Task 1 — `time_reverse_signal(x)`

Return the time-reversed signal $x[-n]$. Since the stored index axis $n = -8 \dots 8$ is symmetric about 0, reversing the array is sufficient here — but state in a comment *why* this works, and what would break if the axis were asymmetric.

Task 2 — `odd_even_decomposition(x)`

Return two arrays: the **odd** component first, then the **even** component. Use $x_e[n] = \frac{1}{2}(x[n] + x[-n])$ and $x_o[n] = \frac{1}{2}(x[n] - x[-n])$. You *must* call your `time_reverse_signal(...)` inside this function.

Checks to include (worth marks — graders love these):

- $x_e[-n] == x_e[n]$ (even part is truly even)
- $x_o[-n] == -x_o[n]$ (odd part is truly odd)
- $x_e[n] + x_o[n] == x[n]$ (reconstruction)

Task 1 (time reversal)	3
Task 2 (even–odd decomposition)	5
Plots + verification of the three checks	2

Common mistake to avoid: an accidental extra factor (e.g. dividing by 2 twice) in the odd term. If $x_e + x_o$ does not reconstruct x exactly, your coefficients are wrong.

Question Set 3 — Time Scaling with Interpolation

Total: 10 Marks | Suggested Time: 35 min

Task 1 — `time_scale_signal(x, k)`

Return the up-scaled signal $x[n/k]$ for positive integer k . Where n/k is an integer, copy the corresponding sample; where n/k is not an integer, **set the intermediate sample to 0**.

Task 2 — `time_scale_signal_interpolate(x, k)`

Same as Task 1, but each intermediate sample (non-integer n/k) takes the **average of the two original samples it lies between** — i.e. average of x at $\text{floor}(n/k)$ and $\text{ceil}(n/k)$. Example: for $k = 3$, the sample at $n = -1$ becomes the average of $x[0]$ and $x[-1]$.

Bonus: Fully vectorized, no explicit loops (hint: `np.floor`, `np.ceil`, boolean masks, `np.where`).

Task 1 ($x[n/k]$, zeros between)	6
Task 2 ($x[n/k]$ with averaging)	4
Bonus (vectorized, no loops)	2

Edge cases to test: $k = 1$ must return the original unchanged in both functions; confirm that with $k = 1$, $\text{floor}(n/k) == \text{ceil}(n/k)$ so no false averaging occurs at exact samples.

Question Set 4 — Combined Transformation $x[\alpha n + \beta]$

Total: 10 Marks | Suggested Time: 35 min

This question generalizes shifting, reversal, and scaling into one function.

Task 1 — `transform(x, alpha, beta)`

Given the stored signal x on $n = -8 \dots 8$, return the signal $y[n] = x[\alpha n + \beta]$ evaluated on the same axis $n = -8 \dots 8$. Here α is a nonzero integer (may be negative, giving reversal) and β is an integer. For each output index n , compute the argument $\alpha n + \beta$; if it is an integer inside $-8 \dots 8$, copy that sample, otherwise output 0.

Task 2 — demonstrate composition order

Using your function, produce $y[n] = x[-2n + 1]$ two ways: (a) in a single call to `transform`; (b) by composing simpler operations (shift, then reverse, then scale) and show they agree. Then produce $x[-2n + 1]$ and $x[-2(n + 1)]$ and explain, in a comment, why the order of shift and scale changes the result.

Task 1 (general transform)	5
Task 2 (composition + order explanation)	3
Plots of original and transformed signals	2

Key idea: evaluating x at the argument $\alpha n + \beta$ directly (rather than chaining array operations) sidesteps operation-ordering bugs entirely.

Question Set 5 — Phase Shift vs. Time Shift in a Sinusoid

Total: 10 Marks | Suggested Time: 30 min

Work with the discrete sinusoid $x[n] = A \cdot \cos(\Omega_0 n + \phi)$ on $n = -20 \dots 20$.

Task 1 — `sinusoid(n, A, Omega0, phi)`

Return $A \cdot \cos(\Omega_0 n + \phi)$ as a NumPy array.

Task 2 — `time_shift_sinusoid(n, A, Omega0, phi, n0)`

Return the sinusoid shifted by integer n_0 , i.e. $x[n - n_0]$.

Task 3 — `phase_change_sinusoid(n, A, Omega0, phi, phi0)`

Return the sinusoid with its phase changed by ϕ_0 . State your sign convention in a comment and be consistent.

Task 4 — **demonstrate the relationship**

(a) Show that a time shift by n_0 equals a phase change by $\phi_0 = \Omega_0 n_0$ (MSE should be ≈ 0). (b) Given an arbitrary phase change ϕ_0 , search integer shifts to find the best match, and explain why the match is generally *not* exact (only phase changes equal to $\Omega_0 \times \text{integer}$ have an exact discrete-time-shift equivalent).

Implementing $x[n]$	1
Time shift	2
Phase change	2
Time shift $\rightarrow$ phase change demo	2
Phase change $\rightarrow$ time shift demo	3

Question Set 6 — Energy, Power, and System Properties

Total: 10 Marks | Suggested Time: 35 min

Part A — Energy and Power

Implement `energy(x)` returning $\sum |x[n]|^2$ over the stored samples, and `average_power(x)` returning the energy divided by the number of samples. Verify on the sample signal that energy = sum of squared values.

Part B — Classify systems

For each system below, write it as a function `system(x)` and classify it as **Linear** / **Nonlinear**, **Time-invariant** / **Time-varying**, and **Causal** / **Non-causal**. Justify each answer briefly, and confirm computationally.

- $y[n] = x[n] - x[n-1]$
- $y[n] = n \cdot x[n]$
- $y[n] = x[n]^2$
- $y[n] = x[n] + 5$
- $y[n] = \frac{1}{2}(x[n-1] + x[n] + x[n+1])$ (centered average)
- $y[n] = x[-n]$

Deliverable: a filled property table (system $\times$ three properties) plus one-line justifications. Bonus for computational verification of each verdict.

Energy + average power	2
Property classification table (6 systems)	6
Computational verification of verdicts	2

Traps deliberately included: $n \cdot x[n]$ is linear but time-varying; $x[n]^2$ is nonlinear but time-invariant; $x[n]+5$ fails linearity ($T\{0\} \neq 0$); the centered average is non-causal (needs $x[n+1]$); $x[-n]$ is non-causal.

Appendix — Property Classification Answer Key (Set 6, Part B)

System	Linear?	Time-Inv?	Causal?
$y[n] = x[n] - x[n-1]$	Yes	Yes	Yes
$y[n] = n \cdot x[n]$	Yes	No	Yes
$y[n] = x[n]^2$	No	Yes	Yes
$y[n] = x[n] + 5$	No	Yes	Yes
$y[n] = \frac{1}{2}(x[n-1] + x[n] + x[n+1])$	Yes	Yes	No
$y[n] = x[-n]$	Yes	No	No

Quick justifications:

- **$x[n] - x[n-1]$** : a fixed linear combination of present and past — linear, time-invariant, causal.
- **$n \cdot x[n]$** : scaling by a known index-dependent coefficient is linear, but the coefficient depends on absolute time, so shifting the input then processing differs from processing then shifting — time-varying. Uses only $x[n]$, so causal.
- **$x[n]^2$** : squaring introduces a cross term under superposition — nonlinear. But it treats every sample identically regardless of position — time-invariant, and causal.
- **$x[n] + 5$** : feeding the zero signal gives 5, not 0, so $T\{0\} \neq 0$ — nonlinear. The offset is constant in time — time-invariant, causal.
- **centered average**: a fixed linear combination — linear and time-invariant, but it needs $x[n+1]$ (a future sample) — non-causal.
- **$x[-n]$** : linear (fixed reindexing), but reversal needs future values in general and depends on absolute time — time-varying and non-causal.

Prepared as a revision aid. Verify every implementation numerically before relying on it, and be ready to explain each verdict to the instructor.